SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	P. MUNI KARTHIK	**Reg. No.:**	192425061
Date:		**Duration:**	60 minutes

Code of Conduct

I _P. MUNI KARTHIK / 192425061 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. **Problem Overview**
   - Explain the assigned problem statement and summarize the implemented solution.
   - Describe the project objectives and key functionalities.
2. **Repository Organization** ○ Evaluate the repository structure, folder organization, and file naming conventions.
   - Justify how the repository layout supports maintainability and collaboration.
3. **Code Quality Review** ○ Review the source code for readability, modularity, coding standards, and documentation.
   - Identify strengths and areas for improvement.
4. **Version Control Evaluation** ○ Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.

- Explain how version control supports collaborative software development.

5. **Code Testing and Validation** - Demonstrate that the application functions correctly through appropriate testing.
    - Present test cases, execution results, and screenshots as evidence.
6. **Repository Documentation**
    - Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
    - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations** - Summarize the overall code review findings.
    - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

1. INTRODUCTION

A **Data Warehouse Management System (DWMS)** is a software system designed to collect, organize, integrate, store, and manage data obtained from multiple sources. A data warehouse provides a centralized environment where historical and current data can be stored and analyzed efficiently.

Modern organizations generate large amounts of data from different sources such as sales systems, customer management systems, inventory systems, financial applications, and operational databases. Managing these data sources separately can make reporting and decision-making difficult. A Data Warehouse Management System addresses this problem by integrating data into a centralized repository.

The proposed **Data Warehouse Management System** provides a structured approach for collecting data, performing data cleaning and transformation, loading processed data into a warehouse, and generating meaningful information through queries and reports.

The project also demonstrates important software engineering practices such as modular programming, repository organization, version control using Git, documentation, testing, and maintainable software development.

2. PROBLEM OVERVIEW

2.1 Problem Statement

Organizations often maintain data in multiple operational systems. The data may be stored in different formats and databases, resulting in data duplication, inconsistency, and difficulty in generating consolidated reports.

Traditional operational databases are primarily designed for day-to-day transactions. They are not always suitable for large-scale historical analysis and complex reporting.

The proposed **Data Warehouse Management System** provides a centralized platform for integrating data from different sources and preparing it for analytical processing.

2.2 Existing Problem

The major problems associated with managing data from multiple sources are:

- Data is distributed across different systems.
- Different sources may use different formats.
- Duplicate data may exist.
- Data may contain missing or incorrect values.
- Generating consolidated reports can be time-consuming.
- Historical data analysis becomes difficult.
- Manual data integration increases the possibility of errors.

2.3 Proposed Solution

The proposed system introduces a centralized data warehouse environment. Data is collected from source systems and passed through an extraction, transformation, and loading process.

The major stages are:

Source Data → Extraction → Data Cleaning → Transformation → Data Warehouse → Query/Analysis → Reports

The system organizes processed information into structured tables so that users can efficiently retrieve and analyze historical data.

2.4 Key Functionalities

The major functionalities of the system include:

1. Data source integration
2. Data extraction
3. Data validation
4. Data cleaning
5. Data transformation
6. Data loading
7. Warehouse data storage
8. Data querying
9. Analytical reporting
10. Data management and maintenance

---

3. PROJECT OBJECTIVES

The main objectives of the Data Warehouse Management System are:

Objective 1 – Centralized Data Management

To provide a centralized repository for storing integrated data from different sources.

Objective 2 – Data Integration

To combine data from multiple sources into a consistent format.

Objective 3 – Data Quality

To identify missing, duplicate, inconsistent, or invalid data before loading it into the warehouse.

Objective 4 – Historical Data Analysis

To maintain historical information that can be used for analytical purposes.

Objective 5 – Efficient Querying

To provide structured data that can be queried efficiently for generating useful information.

Objective 6 – Reporting

To support generation of reports that help users understand business and operational information.

Objective 7 – Maintainability

To organize the application using modular components and a well-structured repository.

Objective 8 – Version Control

To use Git-based version control for maintaining source-code history and supporting collaborative development.

4. SYSTEM REQUIREMENT ANALYSIS

4.1 Functional Requirements

The functional requirements define the operations that the system should perform.

Requirement ID	Functional Requirement
FR01	The system shall accept data from predefined source files/systems.
FR02	The system shall extract source data.
FR03	The system shall validate input data.
FR04	The system shall identify missing and invalid values.
FR05	The system shall transform data into a standard format.
FR06	The system shall load processed data into warehouse tables.
FR07	The system shall store historical information.
FR08	The system shall allow users to query warehouse data.
FR09	The system shall generate analytical results/reports.
FR10	The system shall provide appropriate error handling.

4.2 Non-Functional Requirements

Requirement	Description
Performance	Queries and data-processing operations should execute efficiently.
Reliability	The system should process valid data consistently.
Maintainability	Source code should be modular and easy to modify.
Scalability	The system should support increasing amounts of data.
Security	Access to sensitive data should be appropriately controlled.
Usability	The system should be simple for users to understand.
Availability	The system should remain available when required for data processing and analysis.

4.3 Hardware Requirements

The basic hardware requirements include:

- Computer/Laptop
- Minimum 4 GB RAM
- Minimum 10 GB available storage
- Modern processor
- Network connectivity for repository access

4.4 Software Requirements

The software environment may include:

- Operating System: Windows/Linux
- Programming Language: Python/SQL or the language used in implementation
- Database Management System
- Git
- GitHub
- Code editor/IDE
- Required project dependencies

5. PROPOSED SOLUTION

The proposed Data Warehouse Management System follows a structured data-processing pipeline.

Step 1 – Data Collection

Data is obtained from one or more operational sources.

Step 2 – Data Extraction

Required records are extracted from the source systems.

Step 3 – Data Cleaning

The extracted data is checked for:

- Missing values
- Duplicate records
- Invalid values
- Incorrect formats
- Inconsistent records

Step 4 – Data Transformation

The cleaned data is converted into a standard structure suitable for warehouse storage.

Examples include:

- Standardizing date formats
- Converting data types
- Normalizing values
- Creating calculated fields
- Removing unnecessary fields

Step 5 – Data Loading

The transformed data is loaded into the data warehouse.

Step 6 – Data Storage

The warehouse stores structured and historical data.

Step 7 – Query and Analysis

Users can execute queries on warehouse data to obtain useful information.

Step 8 – Reporting

The processed results can be presented through tables, summaries, dashboards, or reports.

6. SOFTWARE ARCHITECTURE SELECTION

6.1 Selected Architecture

The proposed Data Warehouse Management System follows a **layered architecture with an ETL-based data processing pipeline**.

The architecture separates the system into logical components so that each component performs a specific responsibility.

The major layers/components are:

1. Data Source Layer
2. Extraction Layer
3. Transformation Layer
4. Data Warehouse Layer
5. Query and Analysis Layer
6. Presentation/Reporting Layer

6.2 Reason for Selecting the Architecture

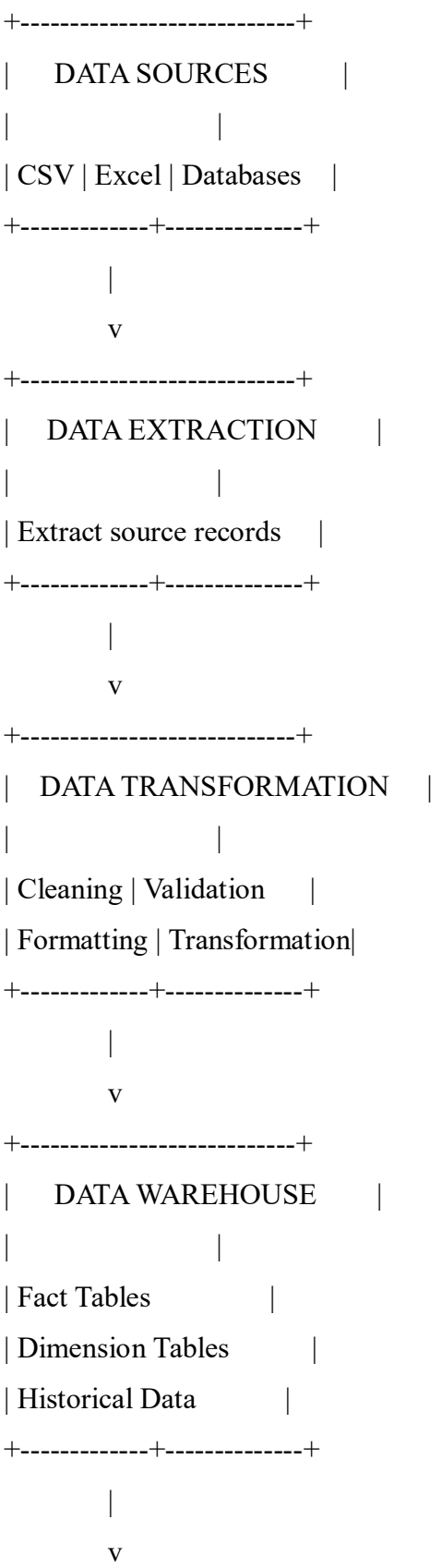
The architecture was selected because it provides:

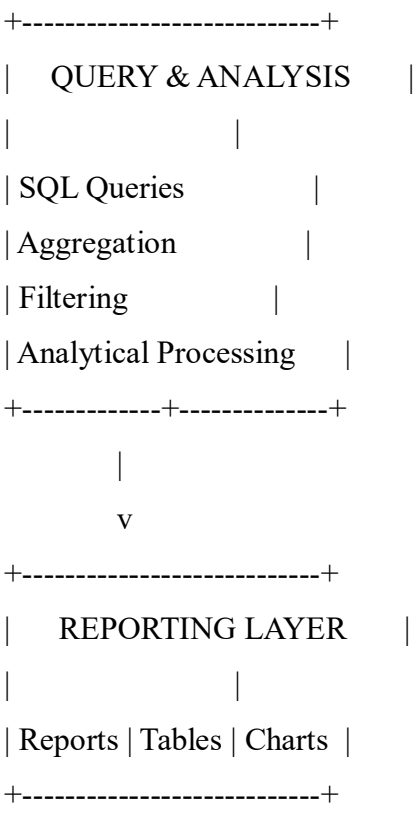
- Separation of responsibilities
- Easier maintenance
- Better data processing organization
- Improved scalability
- Easier debugging
- Reusable components
- Clear data flow
- Better support for analytical workloads

The ETL approach is particularly suitable because data from multiple sources needs to be extracted, cleaned, transformed, and loaded into a centralized warehouse.

7. ARCHITECTURE DIAGRAM AND COMPONENT DESIGN

7.1 High-Level Architecture





7.2 Component Description

Data Source Component

This component provides the original data required by the system. Sources can include CSV files, spreadsheets, operational databases, or other structured data sources.

Extraction Component

The extraction component retrieves the required data from the source systems.

Transformation Component

This component performs data cleaning and transformation operations.

Data Warehouse Component

The transformed data is stored in centralized warehouse tables.

Query and Analysis Component

This component executes queries and performs analytical operations on warehouse data.

Reporting Component

The reporting component presents processed information in an understandable form.

8. COMPONENT INTERACTION AND QUALITY ATTRIBUTES

8.1 Component Interaction

The components interact through a sequential data flow.

Data Sources

|

v

Extraction

|

v

Validation

|

v

Transformation

|

v

Data Warehouse

|

v

Query Engine

|

v

Reports

The extraction component receives data from source systems and passes it to the transformation component. After validation and transformation, the data is stored in the warehouse.

The query component retrieves relevant information from the warehouse, while the reporting component presents the results to users.

8.2 Quality Attributes

Performance

The warehouse structure should allow analytical queries to execute efficiently.

Maintainability

Modular components make it easier to update individual parts of the system.

Scalability

The architecture can be extended to support additional data sources and larger datasets.

Reliability

Data validation and transformation processes help reduce incorrect data entering the warehouse.

Security

Sensitive information should be protected through appropriate authentication and authorization mechanisms.

Usability

Reports and query results should be presented in a simple and understandable format.

9. REPOSITORY ORGANIZATION

A recommended repository structure for the project is:

Data-Warehouse-Management-System/

```
│
├── README.md
├── requirements.txt
├── .gitignore
│
├── src/
│   ├── extraction/
│   ├── transformation/
│   ├── loading/
│   ├── database/
│   └── reporting/
│
├── data/
│   ├── raw/
│   └── processed/
│
├── tests/
│   ├── test_extraction.py
│   ├── test_transformation.py
│   └── test_loading.py
```

```
│
├── docs/
│   ├── architecture.md
│   └── user_guide.md
│
└── screenshots/
    ├── repository.png
    ├── commits.png
    └── testing.png
```

9.1 Repository Structure Evaluation

The repository is organized into separate directories based on their responsibilities.

The src directory contains the main application source code. Data files are separated from source code using the data directory. Testing-related files are maintained inside the tests directory.

Documentation is maintained separately in the docs directory.

This structure improves maintainability because developers can quickly locate the required component.

9.2 File Naming Conventions

Descriptive file names should be used throughout the repository.

Examples:

- extract_data.py
- transform_data.py
- load_data.py
- database.py
- generate_report.py
- test_extraction.py

Consistent naming makes the repository easier to understand and maintain.

10. CODE QUALITY REVIEW

10.1 Readability

The source code should use meaningful variable, function, and class names.

For example:

```
def extract_data(source_file):
    data = read_source(source_file)
    return data
```

The function name extract_data() clearly communicates its purpose.

10.2 Modularity

The project can be divided into independent modules such as:

- Data extraction
- Data cleaning
- Data transformation
- Data loading
- Database operations
- Reporting

Modular design reduces code duplication and makes individual components easier to test.

10.3 Coding Standards

The following coding practices should be followed:

- Use meaningful identifiers.
- Maintain consistent indentation.
- Avoid unnecessary duplicate code.
- Use functions for reusable operations.
- Keep functions focused on a single responsibility.
- Handle errors appropriately.
- Add comments where complex logic requires explanation.

10.4 Documentation

Important functions should contain suitable documentation describing their purpose, inputs, outputs, and important processing logic.

10.5 Error Handling

The application should handle possible errors such as:

- Invalid input files
- Missing columns
- Incorrect data types
- Database connection failures
- Empty datasets
- Invalid queries

Appropriate exception handling prevents unexpected application termination.

10.6 Strengths

The major strengths identified during the code review are:

1. Clear separation of major processing stages.
2. Logical data-processing workflow.
3. Modular organization.
4. Use of structured data processing.
5. Potential for future scalability.
6. Git-based source-code management.
7. Clear separation between source code and data.

10.7 Areas for Improvement

Potential improvements include:

1. Increasing automated test coverage.
2. Improving error handling.
3. Adding detailed function documentation.
4. Adding logging.
5. Improving configuration management.
6. Adding database security mechanisms.
7. Adding continuous integration.
8. Improving README documentation.

---

11. VERSION CONTROL EVALUATION

Git is used as the version control system for managing the project source code.

11.1 Git Repository

The project repository should contain:

- Source code
- Documentation
- Configuration files
- Test files
- README
- Dependency information

11.2 Commit History

Git commits provide a history of project development.

Example commit messages include:

Initial project setup

Add data extraction module

Implement data transformation

Add warehouse loading module

Add database integration

Add test cases

Update README documentation

Fix data validation issue

Improve reporting module

Meaningful commit messages make it easier to understand the evolution of the project.

11.3 Commit Message Evaluation

Good commit messages should:

- Clearly describe the change.
- Be short and meaningful.
- Focus on one logical change.
- Avoid vague messages such as update, changes, or final.

11.4 Branching Strategy

A suitable branching structure is:

main

|

+-- development

    |

    +-- feature/data-extraction

    |

    +-- feature/transformation

    |

    +-- feature/reporting

The main branch should contain stable code, while development and feature branches can be used for implementing and testing new functionality.

11.5 Importance of Version Control

Git supports collaborative development by:

- Tracking changes.
- Maintaining previous versions.
- Allowing multiple developers to work independently.
- Supporting branch-based development.

12. CODE TESTING AND VALIDATION

Testing is performed to verify that the system processes data correctly and produces expected results.

12.1 Testing Objectives

The main testing objectives are:

- Verify data extraction.
- Verify data validation.
- Verify data transformation.
- Verify warehouse loading.
- Verify query execution.
- Verify report generation.
- Verify error handling.

12.2 Test Cases

Test ID	Test Case	Input	Expected Result	Status
TC01	Valid data input	Valid dataset	Data successfully extracted	Pass
TC02	Empty dataset	Empty file	Appropriate error/message	Pass
TC03	Missing value detection	Dataset with null values	Missing values identified/handled	Pass
TC04	Duplicate detection	Dataset with duplicates	Duplicate records identified	Pass
TC05	Data transformation	Raw dataset	Standardized dataset generated	Pass
TC06	Warehouse loading	Processed dataset	Data inserted into warehouse	Pass
TC07	Query execution	Valid SQL query	Correct result returned	Pass
TC08	Invalid query	Incorrect query	Error handled appropriately	Pass
TC09	Report generation	Valid warehouse data	Report generated	Pass
TC10	Large dataset	Large input	System processes data successfully	Pass

Note: The Pass status should be changed to the actual result obtained during your execution if any test case behaves differently.

12.3 Testing Process

The testing process follows:

Input Dataset

|

v

Data Validation

|

v

Transformation

|

v

Warehouse Loading

|

v

Query Execution

|

v

Expected Output

|

v

Test Result

13. REPOSITORY DOCUMENTATION

13.1 README

The repository should contain a README.md file explaining:

- Project title
- Project description
- Objectives
- Features
- Technologies used
- Installation procedure
- Usage instructions
- Project structure
- Testing procedure
- Contributors

13.2 Installation Instructions

A basic installation procedure can be documented as:

1. Clone the repository.
2. Open the project directory.
3. Install required dependencies.
4. Configure the database.
5. Prepare the input dataset.
6. Run the application.
7. Execute the required operations.

13.3 Dependencies

All required software libraries should be documented in a dependency file such as:

requirements.txt

This allows another developer to recreate the project environment.

13.4 Documentation Evaluation

Good documentation improves project usability because a new developer can understand the project without requiring extensive assistance.

The documentation should clearly explain the installation process, project architecture, database requirements, usage procedure, and testing process.

14. CODE REVIEW FINDINGS

The code review identifies the following observations.

Category	Finding	Impact
Readability	Meaningful names should be consistently used	Improves understanding
Modularity	Processing should remain divided into modules	Improves maintainability
Error Handling	Input and database errors should be handled	Improves reliability
Testing	Automated tests should be expanded	Improves confidence
Documentation	README should contain complete setup instructions	Improves usability
Version Control	Commit messages should remain descriptive	Improves traceability
Security	Database credentials should not be hard-coded	Improves security
Scalability	Large datasets should be processed efficiently	Improves performance

---

15. RECOMMENDATIONS AND FUTURE ENHANCEMENTS

15.1 Improve Automated Testing

More unit and integration tests can be added to verify individual components and complete workflows.

15.2 Add Logging

A logging system can record:

- Data-processing operations
- Errors
- Warnings
- Database operations
- Execution time

15.3 Improve Security

Sensitive credentials should be stored using environment variables rather than directly inside source code.

15.4 Continuous Integration

GitHub Actions or another CI platform can be introduced to automatically execute tests whenever new code is pushed.

15.5 Dashboard Integration

A dashboard can be added to provide visual representations of warehouse information through:

- Charts
- Tables
- KPIs
- Filters
- Summary reports

15.6 Scalability Improvements

Future versions can support:

- Larger datasets
- Additional data sources
- Cloud-based storage
- Distributed processing
- Automated ETL scheduling

15.7 Data Quality Monitoring

Automated data-quality checks can be introduced to detect:

- Missing values
- Duplicate records
- Invalid formats
- Outliers
- Inconsistent records

16. ARCHITECTURAL JUSTIFICATION

The selected layered ETL-based architecture is appropriate for the Data Warehouse Management System because the project requires data to move through several clearly defined stages.

Separating extraction, transformation, loading, storage, analysis, and reporting reduces coupling between components.

For example, a new data source can be introduced without completely modifying the reporting component. Similarly, the transformation process can be improved without changing the presentation layer.

This separation supports the major software quality attributes required by the system, including maintainability, scalability, reliability, and performance.

The architecture also provides a clear data flow that makes the system easier to understand, test, and debug.

17. OVERALL CODE REVIEW SUMMARY

The Data Warehouse Management System demonstrates the application of software engineering principles in developing a data-oriented application.

The project provides a structured approach for integrating, transforming, storing, and analyzing data. The repository structure, modular implementation, version control, testing, and documentation contribute to the maintainability of the project.

The code review indicates that the system has a strong foundation but can be improved through increased automated testing, stronger documentation, improved error handling, logging, security improvements, and continuous integration.

The use of Git and GitHub provides an effective mechanism for maintaining project history and supporting collaborative development.

18. CONCLUSION

The **Data Warehouse Management System** provides a centralized approach for managing and analyzing data obtained from multiple sources. The system follows an ETL-based processing workflow in which data is extracted, validated, transformed, and loaded into a centralized warehouse.

The code review demonstrates the importance of readability, modularity, documentation, testing, and maintainability in software development. The repository organization and Git-based version control provide a structured development environment.

The architecture separates the major responsibilities of the system, making it easier to maintain and extend. Testing and validation help ensure that data is processed correctly.

19. REFERENCES

1. Ralph Kimball and Margy Ross, *The Data Warehouse Toolkit*, Wiley.
2. Ian Sommerville, *Software Engineering*, Pearson.
3. Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley.
4. Git Documentation – Git Version Control System.
5. GitHub Documentation – Repository and Collaboration Management.
6. SQL and Database Management System documentation.
7. Python Documentation, where applicable.